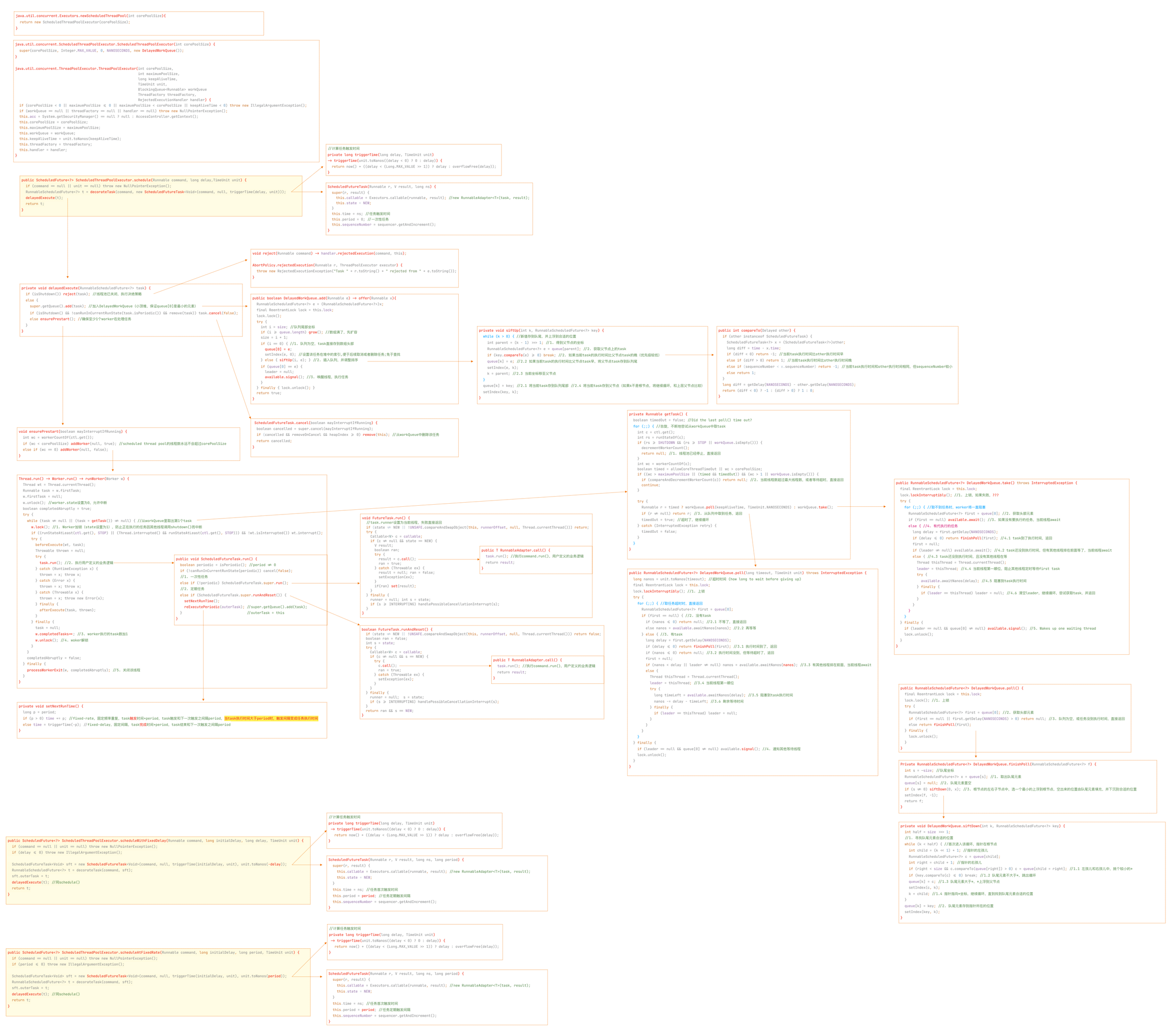

```java
java.util.concurrent.Executors.newScheduledThreadPool(int corePoolSize){
  return new ScheduledThreadPoolExecutor(corePoolSize);
}
```

```java
java.util.concurrent.ScheduledThreadPoolExecutor.ScheduledThreadPoolExecutor(int corePoolSize) {
  super(corePoolSize, Integer.MAX_VALUE, 0, NANOSECONDS, new DelayedWorkQueue());
}

java.util.concurrent.ThreadPoolExecutor.ThreadPoolExecutor(int corePoolSize,
                                                           int maximumPoolSize,
                                                           long keepAliveTime,
                                                           TimeUnit unit,
                                                           BlockingQueue<Runnable> workQueue
                                                           ThreadFactory threadFactory,
                                                           RejectedExecutionHandler handler) {
  if (corePoolSize < 0 || maximumPoolSize ≤ 0 || maximumPoolSize < corePoolSize || keepAliveTime < 0) throw new IllegalArgumentException();
  if (workQueue == null || threadFactory == null || handler == null) throw new NullPointerException();
  this.acc = System.getSecurityManager() == null ? null : AccessController.getContext();
  this.corePoolSize = corePoolSize;
  this.maximumPoolSize = maximumPoolSize;
  this.workQueue = workQueue;
  this.keepAliveTime = unit.toNanos(keepAliveTime);
  this.threadFactory = threadFactory;
  this.handler = handler;
}
```

```java
public ScheduledFuture<?> ScheduledThreadPoolExecutor.schedule(Runnable command, long delay,TimeUnit unit) {
  if (command == null || unit == null) throw new NullPointerException();
  RunnableScheduledFuture<?> t = decorateTask(command, new ScheduledFutureTask<Void>(command, null, triggerTime(delay, unit)));
  delayedExecute(t);
  return t;
}
```

```java
//计算任务触发时间
private long triggerTime(long delay, TimeUnit unit)
→ triggerTime(unit.toNanos((delay < 0) ? 0 : delay)) {
  return now() + ((delay < (Long.MAX_VALUE >> 1)) ? delay : overflowFree(delay));
}
```

```java
ScheduledFutureTask(Runnable r, V result, long ns) {
  super(r, result) {
    this.callable = Executors.callable(runnable, result); //new RunnableAdapter<T>(task, result);
    this.state = NEW;
  }
  this.time = ns; //任务触发时间
  this.period = 0; //一次性任务
  this.sequenceNumber = sequencer.getAndIncrement();
}
```

```java
private void delayedExecute(RunnableScheduledFuture<?> task) {
  if (isShutdown()) reject(task); //线程池已关闭，执行决绝策略
  else {
    super.getQueue().add(task); //加入DelayedWorkQueue (小顶堆，保证queue[0]是最小的元素)
    if (isShutdown() && !canRunInCurrentRunState(task.isPeriodic()) && remove(task)) task.cancel(false);
    else ensurePrestart(); //确保至少1个worker在处理任务
  }
}
```

```java
void reject(Runnable command) → handler.rejectedExecution(command, this);

AbortPolicy.rejectedExecution(Runnable r, ThreadPoolExecutor executor) {
  throw new RejectedExecutionException("Task " + r.toString() + " rejected from " + e.toString());
}
```

```java
public boolean DelayedWorkQueue.add(Runnable e) → offer(Runnable x){
  RunnableScheduledFuture<?> e = (RunnableScheduledFuture<?>)x;
  final ReentrantLock lock = this.lock;
  lock.lock();
  try {
    int i = size; //队列尾部坐标
    if (i ≥ queue.length) grow(); //数组满了，先扩容
    size = i + 1;
    if (i == 0) { //1. 队列为空，task直接存到数组头部
      queue[0] = e;
      setIndex(e, 0); //设置该任务在堆中的索引,便于后续取消或者删除任务;免于查找
    } else { siftUp(i, e); } //2. 插入队列，并调整排序
    if (queue[0] == e) {
      leader = null;
      available.signal(); //3. 唤醒线程，执行任务
    }
  } finally { lock.unlock(); }
  return true;
}
```

```java
private void siftUp(int k, RunnableScheduledFuture<?> key) {
  while (k > 0) { //新值存到队尾，并上浮到合适的位置
    int parent = (k - 1) >>> 1; //1. 得到父节点的坐标
    RunnableScheduledFuture<?> e = queue[parent]; //2. 获取父节点上的task
    if (key.compareTo(e) ≥ 0) break; //2. 如果当前task的执行时间比父节点task的晚 (优先级较低)
    queue[k] = e; //2.2 如果当前task的执行时间比父节点task早，将父节点task存到队列尾
    setIndex(e, k);
    k = parent; //2.3 当前坐标移至父节点
  }
  queue[k] = key; //2.1 将当前task存到队列尾部 //2.4 将当前task存到父节点 (如果k不是根节点，将继续循环，和上层父节点比较)
  setIndex(key, k);
}
```

```java
public int compareTo(Delayed other) {
  if (other instanceof ScheduledFutureTask) {
    ScheduledFutureTask<?> x = (ScheduledFutureTask<?>)other;
    long diff = time - x.time;
    if (diff < 0) return -1; //当前task执行时间比other执行时间早
    else if (diff > 0) return 1; //当前task执行时间比other执行时间晚
    else if (sequenceNumber < x.sequenceNumber) return -1; //当前task执行时间和other执行时间相同，但sequenceNumber较小
    else return 1;
  }
  long diff = getDelay(NANOSECONDS) - other.getDelay(NANOSECONDS);
  return (diff < 0) ? -1 : (diff > 0) ? 1 : 0;
}
```

```java
ScheduledFutureTask.cancel(boolean mayInterruptIfRunning) {
  boolean cancelled = super.cancel(mayInterruptIfRunning);
  if (cancelled && removeOnCancel && heapIndex ≥ 0) remove(this); //从workQueue中删除该任务
  return cancelled;
}
```

```java
void ensurePrestart(boolean mayInterruptIfRunning) {
  int wc = workerCountOf(ctl.get());
  if (wc < corePoolSize) addWorker(null, true); //scheduled thread pool的线程数永远不会超过corePoolSize
  else if (wc == 0) addWorker(null, false);
}
```

```java
Thread.run() → Worker.run() → runWorker(Worker w) {
  Thread wt = Thread.currentThread();
  Runnable task = w.firstTask;
  w.firstTask = null;
  w.unlock(); //worker.state设置为0，允许中断
  boolean completedAbruptly = true;
  try {
    while (task ≠ null || (task = getTask()) ≠ null) { //从workQueue里取出第1个task
      w.lock(); //1. Worker加锁 (state设置为1) ，防止正在执行的任务因其他线程调用shutdown()而中断
      if ((runStateAtLeast(ctl.get(), STOP) || (Thread.interrupted() && runStateAtLeast(ctl.get(), STOP))) && !wt.isInterrupted()) wt.interrupt();
      try {
        beforeExecute(wt, task);
        Throwable thrown = null;
        try {
          task.run(); //2. 执行用户定义的业务逻辑
        } catch (RuntimeException x) {
          thrown = x; throw x;
        } catch (Error x) {
          thrown = x; throw x;
        } catch (Throwable x) {
          thrown = x; throw new Error(x);
        } finally {
          afterExecute(task, thrown);
        }
      } finally {
        task = null;
        w.completedTasks++; //3. worker执行的task数加1
        w.unlock(); //4. woker解锁
      }
    }
    completedAbruptly = false;
  } finally {
    processWorkerExit(w, completedAbruptly); //5. 关闭该线程
  }
}
```

```java
private Runnable getTask() {
  boolean timedOut = false; //Did the last poll() time out?
  for (;;) { //自旋，不断地尝试从workQueue中取task
    int c = ctl.get();
    int rs = runStateOf(c);
    if (rs ≥ SHUTDOWN && (rs ≥ STOP || workQueue.isEmpty())) {
      decrementWorkerCount();
      return null; //1. 线程池已经停止，直接返回
    }
    int wc = workerCountOf(c);
    boolean timed = allowCoreThreadTimeOut || wc > corePoolSize;
    if ((wc > maximumPoolSize || (timed && timedOut)) && (wc > 1 || workQueue.isEmpty())) {
      if (compareAndDecrementWorkerCount(c)) return null; //2. 当前线程数超过最大线程数，或者等待超时，直接返回
      continue;
    }

    try {
      Runnable r = timed ? workQueue.poll(keepAliveTime, TimeUnit.NANOSECONDS) : workQueue.take();
      if (r ≠ null) return r; //3. 从队列中取到任务，返回
      timedOut = true; //超时了，继续循环
    } catch (InterruptedException retry) {
      timedOut = false;
    }
  }
}
```

```java
public RunnableScheduledFuture<?> DelayedWorkQueue.take() throws InterruptedException {
  final ReentrantLock lock = this.lock;
  lock.lockInterruptibly(); //1. 上锁，如果失败，???
  try {
    for (;;) { //取不到任务时，worker将一直阻塞
      RunnableScheduledFuture<?> first = queue[0]; //2. 获取头部元素
      if (first == null) available.await(); //3. 如果没有要执行的任务，当前线程await
      else { //4. 有代执行的任务
        long delay = first.getDelay(NANOSECONDS);
        if (delay ≤ 0) return finishPoll(first); //4.1 task到了执行时间，返回
        first = null;
        if (leader ≠ null) available.await(); //4.2 task还没到执行时间，但有其他线程排在前面等了，当前线程await
        else { //4.3 task还没到执行时间，且没有其他线程在等
          Thread thisThread = Thread.currentThread();
          leader = thisThread; //4.4 当前线程第一顺位，阻止其他线程定时等待first task
          try {
            available.awaitNanos(delay); //4.5 阻塞到task执行时间
          } finally {
            if (leader == thisThread) leader = null; //4.6 清空leader，继续循环，尝试获取task，并返回
          }
        }
      }
    }
  } finally {
    if (leader == null && queue[0] ≠ null) available.signal(); //5. Wakes up one waiting thread
    lock.unlock();
  }
}
```

```java
public RunnableScheduledFuture<?> DelayedWorkQueue.poll(long timeout, TimeUnit unit) throws InterruptedException {
  long nanos = unit.toNanos(timeout); //超时时间 (how long to wait before giving up)
  final ReentrantLock lock = this.lock;
  lock.lockInterruptibly(); //1. 上锁
  try {
    for (;;) { //取任务超时时，直接返回
      RunnableScheduledFuture<?> first = queue[0];
      if (first == null) { //2. 没有task
        if (nanos ≤ 0) return null; //2.1 不等了，直接返回
        else nanos = available.awaitNanos(nanos); //2.2 再等等
      } else { //3. 有task
        long delay = first.getDelay(NANOSECONDS);
        if (delay ≤ 0) return finishPoll(first); //3.1 执行时间到了，返回
        if (nanos ≤ 0) return null; //3.2 执行时间没到，但等待超时了，返回
        first = null;
        if (nanos < delay || leader ≠ null) nanos = available.awaitNanos(nanos); //3.3 有其他线程排在前面，当前线程await
        else {
          Thread thisThread = Thread.currentThread();
          leader = thisThread; //3.4 当前线程第一顺位
          try {
            long timeLeft = available.awaitNanos(delay); //3.5 阻塞到task执行时间
            nanos -= delay - timeLeft; //3.6 剩余等待时间
          } finally {
            if (leader == thisThread) leader = null;
          }
        }
      }
    }
  } finally {
    if (leader == null && queue[0] ≠ null) available.signal(); //4. 通知其他等待线程
    lock.unlock();
  }
}
```

```java
public RunnableScheduledFuture<?> DelayedWorkQueue.poll() {
  final ReentrantLock lock = this.lock;
  lock.lock(); //1. 上锁
  try {
    RunnableScheduledFuture<?> first = queue[0]; //2. 获取头部元素
    if (first == null || first.getDelay(NANOSECONDS) > 0) return null; //3. 队列为空，或任务没到执行时间，直接返回
    else return finishPoll(first);
  } finally {
    lock.unlock();
  }
}
```

```java
Private RunnableScheduledFuture<?> DelayedWorkQueue.finishPoll(RunnableScheduledFuture<?> f) {
  int s = --size; //队尾坐标
  RunnableScheduledFuture<?> x = queue[s]; //1. 取出队尾元素
  queue[s] = null; //2. 队尾元素置空
  if (s ≠ 0) siftDown(0, x); //3. 根节点的左右子节点中，选一个最小的上浮到根节点，空出来的位置由队尾元素填充，并下沉到合适的位置
  setIndex(f, -1);
  return f;
}
```

```java
private void DelayedWorkQueue.siftDown(int k, RunnableScheduledFuture<?> key) {
  int half = size >>> 1;
  //1. 寻找队尾元素合适的位置
  while (k < half) { //首次进入该循环，指针在根节点
    int child = (k << 1) + 1; //指针的左孩儿
    RunnableScheduledFuture<?> c = queue[child];
    int right = child + 1; //指针的右孩儿
    if (right < size && c.compareTo(queue[right]) > 0) c = queue[child = right]; //1.1 左孩儿和右孩儿中，挑个较小的*
    if (key.compareTo(c) ≤ 0) break; //1.2 队尾元素不大于*，跳出循环
    queue[k] = c; //1.3 队尾元素大于*，*上浮到父节点
    setIndex(c, k);
    k = child; //1.4 指针指向*坐标，继续循环，直到找到队尾元素合适的位置
  }
  queue[k] = key; //2. 队尾元素存到指针所在的位置
  setIndex(key, k);
}
```

```java
public void ScheduledFutureTask.run() {
  boolean periodic = isPeriodic(); //period ≠ 0
  if (!canRunInCurrentRunState(periodic)) cancel(false);
  //1. 一次性任务
  else if (!periodic) ScheduledFutureTask.super.run();
  //2. 定期任务
  else if (ScheduledFutureTask.super.runAndReset()) {
    setNextRunTime();
    reExecutePeriodic(outerTask); //super.getQueue().add(task);
  }                               //outerTask = this
}
```

```java
void FutureTask.run() {
  //task.runner设置为当前线程，失败直接返回
  if (state ≠ NEW || !UNSAFE.compareAndSwapObject(this, runnerOffset, null, Thread.currentThread())) return;
  try {
    Callable<V> c = callable;
    if (c ≠ null && state == NEW) {
      V result;
      boolean ran;
      try {
        result = c.call();
        ran = true;
      } catch (Throwable ex) {
        result = null; ran = false;
        setException(ex);
      }
      if(ran) set(result);
    }
  } finally {
    runner = null; int s = state;
    if (s ≥ INTERRUPTING) handlePossibleCancellationInterrupt(s);
  }
}
```

```java
public T RunnableAdapter.call() {
  task.run(); //执行command.run()，用户定义的业务逻辑
  return result;
}
```

```java
boolean FutureTask.runAndReset() {
  if (state ≠ NEW || !UNSAFE.compareAndSwapObject(this, runnerOffset, null, Thread.currentThread())) return false;
  boolean ran = false;
  int s = state;
  try {
    Callable<V> c = callable;
    if (c ≠ null && s == NEW) {
      try {
        c.call();
        ran = true;
      } catch (Throwable ex) {
        setException(ex);
      }
    }
  } finally {
    runner = null;  s = state;
    if (s ≥ INTERRUPTING) handlePossibleCancellationInterrupt(s);
  }
  return ran && s == NEW;
}
```

```java
public T RunnableAdapter.call() {
  task.run(); //执行command.run()，用户定义的业务逻辑
  return result;
}
```

```java
private void setNextRunTime() {
  long p = period;
  if (p > 0) time += p; //fixed-rate, 固定频率重复, task触发时间+period, task触发和下一次触发之间隔period, 当task执行时间大于period时，触发间隔变成任务执行时间
  else time = triggerTime(-p); //fixed-delay, 固定间隔, task完成时间+period, task结束和下一次触发之间隔period
}
```

```java
public ScheduledFuture<?> ScheduledThreadPoolExecutor.scheduleWithFixedDelay(Runnable command, long initialDelay, long delay, TimeUnit unit) {
  if (command == null || unit == null) throw new NullPointerException();
  if (delay ≤ 0) throw new IllegalArgumentException();

  ScheduledFutureTask<Void> sft = new ScheduledFutureTask<Void>(command, null, triggerTime(initialDelay, unit), unit.toNanos(-delay));
  RunnableScheduledFuture<?> t = decorateTask(command, sft);
  sft.outerTask = t;
  delayedExecute(t); //同schedule()
  return t;
}
```

```java
//计算任务触发时间
private long triggerTime(long delay, TimeUnit unit)
→ triggerTime(unit.toNanos((delay < 0) ? 0 : delay)) {
  return now() + ((delay < (Long.MAX_VALUE >> 1)) ? delay : overflowFree(delay));
}
```

```java
ScheduledFutureTask(Runnable r, V result, long ns, long period) {
  super(r, result) {
    this.callable = Executors.callable(runnable, result); //new RunnableAdapter<T>(task, result);
    this.state = NEW;
  }
  this.time = ns; //任务首次触发时间
  this.period = period; //任务定期触发间隔
  this.sequenceNumber = sequencer.getAndIncrement();
}
```

```java
public ScheduledFuture<?> ScheduledThreadPoolExecutor.scheduleAtFixedRate(Runnable command, long initialDelay, long period, TimeUnit unit) {
  if (command == null || unit == null) throw new NullPointerException();
  if (period ≤ 0) throw new IllegalArgumentException();

  ScheduledFutureTask<Void> sft = new ScheduledFutureTask<Void>(command, null, triggerTime(initialDelay, unit), unit.toNanos(period));
  RunnableScheduledFuture<?> t = decorateTask(command, sft);
  sft.outerTask = t;
  delayedExecute(t); //同schedule()
  return t;
}
```

```java
//计算任务触发时间
private long triggerTime(long delay, TimeUnit unit)
→ triggerTime(unit.toNanos((delay < 0) ? 0 : delay)) {
  return now() + ((delay < (Long.MAX_VALUE >> 1)) ? delay : overflowFree(delay));
}
```

```java
ScheduledFutureTask(Runnable r, V result, long ns, long period) {
  super(r, result) {
    this.callable = Executors.callable(runnable, result); //new RunnableAdapter<T>(task, result);
    this.state = NEW;
  }
  this.time = ns; //任务首次触发时间
  this.period = period; //任务定期触发间隔
  this.sequenceNumber = sequencer.getAndIncrement();
}
```